
React

Interview Questions & Answers | 24 questions | Building user interfaces

Contents

React Fundamentals (Q1-Q5)

Hooks (Q6-Q12)

Component Design & Performance (Q13-Q18)

Rendering, Lifecycle & Architecture (Q19-Q24)

React Fundamentals

Q1. What is React and what problem does it solve?

React is a JavaScript library for building user interfaces out of reusable components. It solves the problem of efficiently keeping the UI in sync with changing data by using a declarative model - you describe what the UI should look like for a given state, and React figures out how to update the DOM to match.

Q2. What is the Virtual DOM and how does it improve performance?

The Virtual DOM is an in-memory, lightweight representation of the real DOM. When state changes, React builds a new virtual tree, diffs it against the previous one (reconciliation), and computes the minimal set of real DOM operations needed to apply the difference. This batches and minimizes expensive direct DOM manipulation.

Q3. What is JSX?

JSX is a syntax extension that lets you write HTML-like markup inside JavaScript. It's not understood by browsers directly - it's compiled (usually via Babel) into `React.createElement()` calls that produce plain JavaScript objects describing the UI.

```
// JSX
const el = <h1 className="title">Hello</h1>;

// Compiles to
const el = React.createElement('h1', { className: 'title' }, 'Hello');
```

Q4. What is the difference between a controlled and an uncontrolled component?

A controlled component's form value is driven entirely by React state - the input's value prop is set from state and updated via `onChange`. An uncontrolled component manages its own state internally in the DOM, and React reads its value on demand using a ref.

```
// Controlled
const [value, setValue] = useState('');
<input value={value} onChange={e => setValue(e.target.value)} />

// Uncontrolled
const inputRef = useRef();
<input ref={inputRef} defaultValue="initial" />
```

Q5. What are keys in React lists and why are they important?

Keys give React a stable identity for each item in a list so it can correctly match elements between renders during reconciliation - deciding what to update, add, or remove. Using array indexes as keys is discouraged when the list can reorder or filter, because it can cause state to be associated with the wrong item.

Hooks

Q6. What is the difference between useState and useReducer?

useState is best for simple, independent pieces of state. useReducer is preferable when state logic is complex, involves multiple sub-values, or when the next state depends on the previous one in non-trivial ways - it centralizes update logic in a single reducer function, similar to Redux.

```
function reducer(state, action) {
  switch (action.type) {
    case 'increment': return { count: state.count + 1 };
    case 'reset': return { count: 0 };
    default: throw new Error('Unknown action');
  }
}
const [state, dispatch] = useReducer(reducer, { count: 0 });
```

Q7. Explain useEffect and its dependency array.

useEffect lets you run side effects (data fetching, subscriptions, manual DOM changes) after render. The dependency array controls when it re-runs: omitted means every render, [] means only on mount, and [a, b] means it re-runs whenever a or b changes. A cleanup function returned from the effect runs before the next effect and on unmount.

```
useEffect(() => {
  const id = setInterval(() => setTick(t => t + 1), 1000);
  return () => clearInterval(id); // cleanup
}, []); // run once on mount
```

Q8. What is the difference between useEffect and useLayoutEffect?

useEffect runs asynchronously after the browser has painted the screen. useLayoutEffect runs synchronously after DOM mutations but before the browser paints, blocking visual updates until it completes. Use useLayoutEffect only when you need to read layout (like measuring an element) and synchronously adjust the DOM to avoid a visible flicker.

Q9. What is the difference between useMemo and useCallback?

Both memoize a value between renders based on a dependency array, avoiding unnecessary recomputation. useMemo memoizes the result of a computation (any value). useCallback memoizes a function reference itself, which is useful for preventing unnecessary re-renders of memoized child components that receive that function as a prop.

```
const total = useMemo(() => expensiveSum(items), [items]);
const handleClick = useCallback(() => doSomething(id), [id]);
```

Q10. What are the Rules of Hooks?

React relies on the order hooks are called in to associate state with the correct hook call across renders.

- Only call hooks at the top level - never inside loops, conditions, or nested functions
- Only call hooks from React function components or custom hooks
- Custom hook names must start with 'use' so linters and React can identify them

Q11. What is a custom hook and when would you create one?

A custom hook is a JavaScript function whose name starts with 'use' and that calls other hooks internally, letting you extract and reuse stateful logic across multiple components (e.g. useFetch, useLocalStorage, useDebounce) without duplicating code or adding wrapper components.

```
function useWindowWidth() {
  const [width, setWidth] = useState(window.innerWidth);
  useEffect(() => {
    const onResize = () => setWidth(window.innerWidth);
    window.addEventListener('resize', onResize);
    return () => window.removeEventListener('resize', onResize);
  }, []);
  return width;
}
```

Q12. What does useRef do, beyond accessing DOM nodes?

useRef returns a mutable object with a .current property that persists across renders without causing a re-render when changed. Besides holding DOM references, it's commonly used to store any mutable value (like a timer ID or previous prop value) that shouldn't trigger a re-render.

Component Design & Performance

Q13. What is prop drilling and how can you avoid it?

Prop drilling is passing data through many layers of components that don't need it themselves, just to reach a deeply nested child. It can be avoided with the Context API for cross-cutting data, component composition (passing children/elements as props), or a state management library for complex global state.

Q14. What is the Context API and when should you use it?

Context provides a way to pass data through the component tree without explicitly passing props at every level. It's best for low-frequency-update, broadly-needed data (theme, authenticated user, locale). For high-frequency updates it can cause unnecessary re-renders of all consumers, so it's not a full replacement for dedicated state management in performance-sensitive cases.

Q15. How does React.memo work and when should you use it?

React.memo is a higher-order component that wraps a component and skips re-rendering it if its props haven't changed (shallow comparison), similar to PureComponent for class components. It's useful for pure components that render often with the same props, but adds overhead, so it shouldn't be applied blindly everywhere.

Q16. What causes unnecessary re-renders in React and how do you prevent them?

Common causes include parent re-renders cascading to children, new object/array/function literals created on every render (breaking reference equality checks), and unstable context values.

- Wrap components in React.memo where appropriate
- Memoize callbacks and derived values with useCallback/useMemo
- Split large components and lift state down, not up, to narrow render scope
- Split context providers so unrelated updates don't re-render everything
- Use React DevTools Profiler to actually measure before optimizing

Q17. What is the difference between a Higher-Order Component (HOC) and a render prop?

Both are patterns for sharing logic between components. An HOC is a function that takes a component and returns a new enhanced component. A render prop is a component that takes a function as a prop (or children) and calls it to determine what to render. Hooks have largely replaced both patterns in modern React code.

Q18. What are error boundaries?

Error boundaries are class components implementing componentDidCatch and/or getDerivedStateFromError that catch JavaScript errors thrown anywhere in their child component tree during rendering, log them, and display a fallback UI instead of crashing the whole app. They cannot catch errors in event handlers, async code, or the boundary component itself.

Rendering, Lifecycle & Architecture

Q19. Describe the lifecycle of a React function component (mount, update, unmount).

On mount, the component function runs, JSX is returned, DOM is committed, then `useEffect` callbacks fire. On update, triggered by new props or state, the function re-runs, React diffs the output, commits changes, then runs effects whose dependencies changed (cleanup of the previous effect runs first). On unmount, cleanup functions from all active effects run.

Q20. What is reconciliation and the diffing algorithm's key assumptions?

Reconciliation is the process React uses to compare a new virtual DOM tree with the previous one to determine the minimal DOM updates needed. It assumes elements of different types produce different trees (so it tears down and rebuilds rather than diffing children), and it uses keys to match list items across renders instead of comparing by position alone.

Q21. What is the difference between Client-Side Rendering (CSR) and Server-Side Rendering (SSR)?

CSR ships a minimal HTML shell and JavaScript bundle to the browser, which then renders the UI and fetches data client-side - simpler but slower initial paint and worse SEO. SSR renders the full HTML on the server for each request and sends complete markup to the browser, improving perceived load time and SEO, then 'hydrates' it with React on the client to make it interactive.

Q22. What is hydration in the context of SSR?

Hydration is the process where React attaches event listeners and internal state to server-rendered HTML on the client, turning static markup into a fully interactive React app, without re-rendering the DOM from scratch (assuming the markup matches).

Q23. What are React Server Components?

React Server Components render exclusively on the server and their output (not component code) is streamed to the client - they can access backend resources directly (databases, file system) and never ship their JavaScript to the browser, reducing bundle size. They're combined with Client Components (marked with 'use client') for interactive parts, as used in frameworks like Next.js's App Router.

Q24. How would you implement code-splitting in a React app?

Use `dynamic import()` together with `React.lazy()` to load a component's code only when it's needed, wrapped in a `Suspense` boundary to show a fallback while it loads. This reduces the initial bundle size and speeds up first load.

```
const Settings = React.lazy(() => import('./Settings'));

<Suspense fallback=<Spinner />>
  <Settings />
</Suspense>
```